BINF6970 - Database: Final Project Documentation

Hang Nguyen

Github repository: https://github.com/hangnguyen259/BINF6970---Database---FinalProject

---

Project overview:

The project's objective is to build a database using TCGA clinical cancer data downloaded from cBioPortal. The goal of the database is that patient IDs and/or sample IDs can be used to find mutation information and the expression information. The mutation information contains variant information, protein change, chromosome position, etc. that is helpful for researching cancer mutation variants. Expression data is raw mRNA expression data; users can retrieve the mRNA expression and decide to either use the raw values or perform normalization on the raw values to use in downstream analyses.

Data sources:

The dataset used in this project is a liver cancer dataset (hepatocellular carcinoma) from TCGA that was downloaded from cBioPortal. The table below shows all the data files available and their descriptions.

File name	Description	Decision
data_mutations.txt (mutation data)	Somatic mutation calls that have gene symbol, Entrez ID, chromosome/position, reference/variant allele, variant classification, protein change, sample barcode, etc.	Used
data_mrna_seq_v2_rsem.txt (expression data)	RNA-seq expression matrix using RSEM-normalized expression values, typically genes by samples.	Used
data_clinical_patient.txt (patient data)	Contains clinical data including patient IDs, age, gender, race, ethnicity, diagnosis, and prognosis	Used
data_clinical_sample.txt (sample data)	Contains patient IDs and sample IDs. Details about cancer type, tissue, and aneuploidy score are provided	Used
data_clinical_supp_hypoxia.txt	Patient IDs and hypoxia scores.	Not used
data_cna.txt	Display discrete copy-number alteration matrix for genes by samples. Values are encoded deletions/amplifications.	Not used
data_gene_panel_matrix.txt	Sequencing gene panels was used for each sample and molecular profile.	Not used
data_genetic_ancestry.txt	Genetic ancestry estimates or ancestry category assignments for patients/samples.	Not used
data_log2_cna.txt	Continuous log2 copy-number ratio matrix	Not used

data_methylation_hm27_hm450_merged.txt	DNA methylation beta-value matrix from Illumina HumanMethylation27 and/or HumanMethylation450 platforms, merged across probes/genes as provided by the study.	Not used
data_armlevel_cna.txt	Contains chromosome arm-level copy-number alteration calls for each sample.	Not used
data_mrna_seq_v2_rsem_zscores_ref_all_samples.txt	Gene-expression z-scores computed relative to all samples in the study.	Not used
data_mrna_seq_v2_rsem_zscores_ref_diploid_samples.txt	Gene-expression z-scores computed relative to copy-number diploid samples, often used to compare expression while reducing CNA-driven baseline effects.	Not used
data_mrna_seq_v2_rsem_zscores_ref_normal_samples.txt	Gene-expression z-scores computed relative to normal samples, when normal samples are available	Not used
data_resource_definition.txt	Definitions of external resources used by data_resource_patient.	Not used
data_resource_patient.txt	Patient-level links/resources, often pointers to external portals, pathology reports, and images.	Not used
data_rppa.txt	Contains Reverse Phase Protein Array (RPPA) protein-expression measurements for TCGA tumor samples.	Not used
data_rppa_zscores.txt	Z-score normalized version of data_rppa.txt	Not used
data_sv.txt	Structural variant calls, such as gene fusions, translocations, inversions, deletions, duplications, or other rearrangements.	Not used
data_timeline_sample_acquisition.txt	Timeline events describing when samples were acquired/collected, often relative to diagnosis or treatment.	Not used
data_timeline_status.txt	Timeline/status events for patients, such as disease status, progression, recurrence, remission, or follow-up events over time	Not used
data_timeline_treatment.txt	Treatment timeline events, such as drug therapy, radiation, surgery, or other interventions with dates/days from diagnosis.	Not used

Not all columns in those 4 files were used to build the database. Here are the breakdown of columns used from each file and the corresponding entity or attribute that used the value of that column in the database.

Data_clinical_patient.txt (patient data)

Column name in the file	Corresponding entity/attribute in the database
PATIENT_ID	patient_id
SEX	gender

RACE	race
ETHNICITY	ethnicity
AGE	age
CANCER_TYPE_ACRONYM	cancer_type
AJCC_PATHOLOGIC_TUMOR_STAGE	stage

Data_clinical_sample.txt (sample data)

Column name in the file	Corresponding entity/attribute in the database
PATIENT_ID	patient_id
SAMPLE_ID	sample_id
TUMOR_TISSUE_SITE	tumor_tissue

Data_mutations.txt (mutation data)

Column name in the file	Corresponding entity/attribute in the database
HUGO_SYMBOL	hugo_symbol
ENTREZ_GENE_ID	entrez_id
CHROMOSOME	chromosome
START_POSITION	start_pos
END_POSITION	end_pos
VARIANT_CLASSIFICATION	variantClassification
VARIANT_TYPE	varian_type
REFERENCE_ALLELE	reference_allele
TUMOR_SEQ_ALLELE1	tumor_allele1
TUMOR_SEQ_ALLELE2	tumor_allele2
dbSNP_RS	dbSNP
TUMOR_SAMPLE_BARCODE	sample_id
HGVSP_SHORT	HGVSp
CODONS	codon_change
POLYPHEN	polyphen_score

Data_mrna_seq_v2_rsem.txt (expression data)

Column name in the file	Corresponding entity/attribute in the database
HUGO_symbol	hugo_symbol
Entrez_ID	entrez_id

TUMOR_SAMPLE_BARCODE columns	Each column contains expression value by gene

Data cleaning decisions

1. **Mutation file (raw: data_mutations.txt -> cleaned: mutation_mapped_fixed_final.tsv)**
   - **Issues:**
     1. In the original mutation file there are columns that have column headers but there is no value in those columns.
     2. There are also cells that have values recorded as "." and this indicates missing values.
     3. Contains HUGO symbols that are outdated or the invalid Entrez ID (Entrez ID = 0) for genes with different HUGO symbols. There were ~3000 rows that have Entrez ID = 0.
     4. After the first mapping, there are 250 genes that have the HUGO symbol but it was not mapped because the matching HUGO symbol could not be located in the reference document.
   - **Solution:**
     1. The first 2 issues are addressed using python data cleaning script (01_clean_data.py). Values recorded as "." are replaced with pd.NA, pandas' unified missing-value marker. This makes missing values more consistent across columns with different data types and helps simplify downstream analysis.
     2. To address this issue, a file with updated HUGO symbol and Entrez ID name 'hgnc_complete_set.txt' was downloaded from genenames.org. The first mapping script (02_map_genes.py) used the downloaded document as a reference to update the HUGO symbol or Entrez ID in the mutation file. After mapping, the number of rows with empty Entrez ID decreased to 250 rows.
     3. After some investigation, there were two rows in the mutation file that have missing Entrez ID but HUGO symbols for these rows could be used to look up EntrezID in the expression file because the expression file has no row with Entrez ID of 0. The second mapping script (03_fix_mutation_from_expression) used the HUGO symbols in the file expression_mapped_final.tsv as reference to fill out the two missing Entrez IDs in the mutation file.
   - **Other decision:** No rows were dropped in the mutation file because all rows have at least one gene identifier (HUGO symbol or Entrez ID) so the mutation data can still be looked up using one of the gene identifiers. The rest of the genes with

Entrez ID value of 0 were replaced with pandas' unified missing-value marker; these genes still have at least the HUGO symbols.

2. Expression file (raw: data_mrna_seq_v2_rsem.txt -> cleaned: expression_mapped_final.tsv)

- **Issues:**
    1. There were outdated or missing HUGO symbols but all genes have Entrez ID.
- **Solutions:**
    1. Outdated or missing HUGO symbols were handled by the first mapping script (02_map_genes.py) that used the file 'hgnc_complete_set.txt' as reference. After updating the symbols, there are 13 rows with empty HUGO symbols but have at least valid Entrez ID so these rows were kept. Entrez IDs of these were used to look up in the mutation file to find matching HUGO symbols but there was no match for those Entrez IDs.
- **Other decisions:** all expression values were kept including expression value of 0. This was to ensure the raw expression data has no bias when using the data in downstream analyses such as normalization, clustering, model training, etc…

3. Patient data and sample data (raw: data_clinical_patient.txt, data_clinical_sample.txt -> cleaned: patient_data_cleaned_final.tsv, sample_data_cleaned_final.tsv)

The cleaning script (01_clean_data.py) was used on the patient file and sample file to check for any duplications, missing data, and clean the data by unifying the null value using pandas' unified missing-value marker. There was no duplication or missing of major data was found in either files. A few missing values in the patient data such as gender or age were observed but because that information does not have a major impact on the quality of the data or potentially affect downstream bioinformatics analyses, therefore, the patients with missing value for gender or age are still kept.

Database design decisions

The database was designed to connect patient basic demographics information such as gender,ethnicity, and race to the molecular data of the patient's tumor sample such as mutation and expression. The design allows for researching how genetic mutations and expression values relate to cancer type, tumor tissue, patient characteristics, and disease stage.

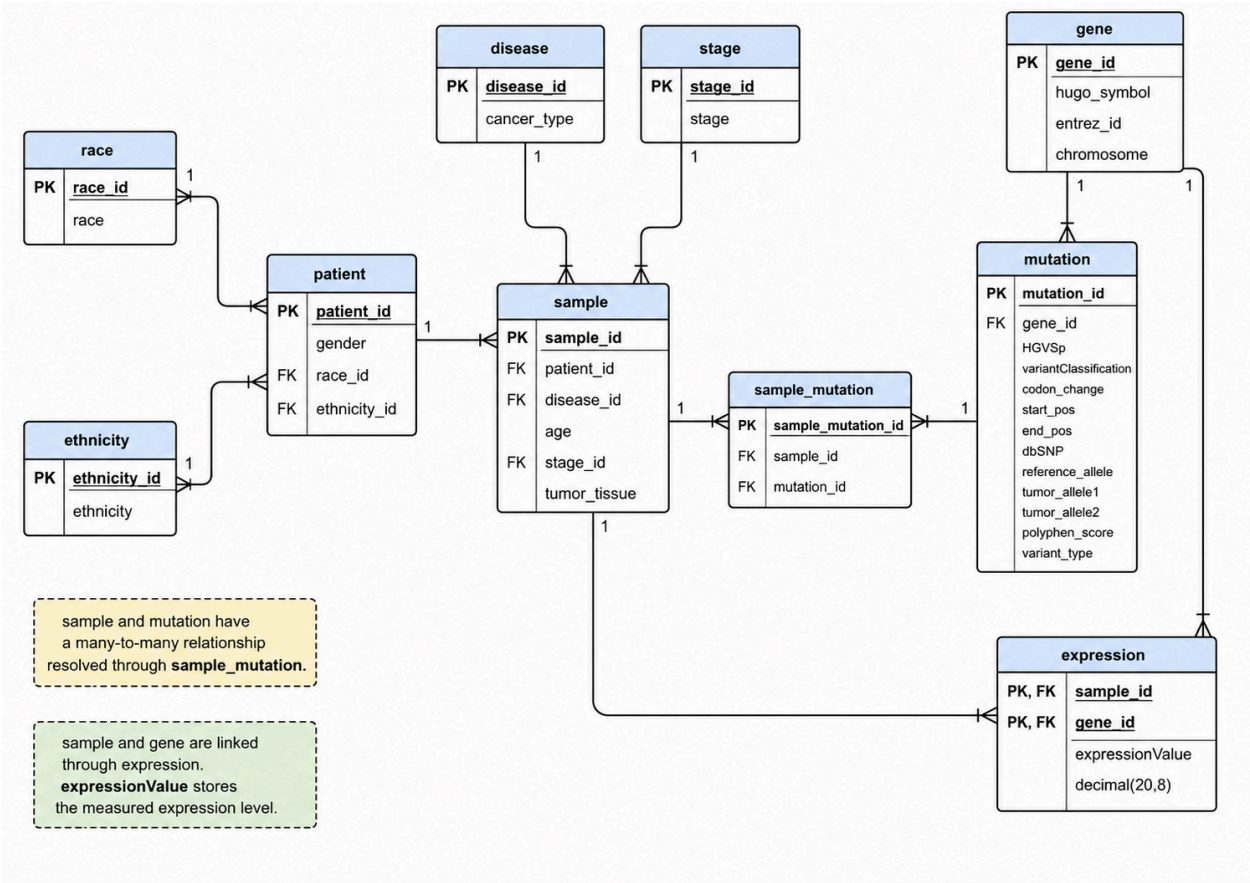

Figure 1. ER diagram of the database

Main entities of the database:

Patient: stores patient identifier and patient demographic information such as gender, race, ethnicity.
Race and ethnicity: these two were made into their own tables to avoid redundancy and help with categories' consistencies.
Sample: patient IDs and their corresponding sample IDs are linked in this table. This table also gives basic disease information such as disease (type of cancer), tumor tissue, and stage of cancer. The attribute 'age' is included in this table because a patient can have multiple diagnoses at different ages in their lifetime, so age is linked to the sample to best describe the sample record.
Stage: In general terms, cancer has 4 main stages (I, II, III, and IV) but the stages can be more specific by including subgroups (A, B, C). In the dataset that was used to build the database, there are some samples that have subgroups recorded. All the different stages observed in the dataset are I, II, III, IIIA, IIIB, IIIC, IV, IVA, IVB. The stage table was made into its own table to reduce redundancy and also to avoid making assumptions that could affect the integrity of the data. An example of such assumptions is grouping entries of the same stage but having different

subgroups together into one stage. Having the stage table separately can help with consistency in case the staging has updates.

Disease: disease was created as a separate table to store standardized cancer type values. This helps reduce data redundancy because the cancer type is stored once and referenced by disease_id in the sample table. It also improves data consistency by preventing spelling differences or duplicate labels for the same disease type.

Mutation: this table stores detailed variant information on the tumor sample. This table is linked to the gene table because each mutation belongs to a specific gene. This design separates gene-level information from mutation-level information which helps reduce redundancy and keeps the database organized.

Sample_mutation: this is a junction table that connects the sample table to the mutation table because a sample can have many mutations and a mutation can be found in multiple samples.. This table represents the many-to-many relationship between tumor samples and mutations. This helps keep the database normalized and avoid multi-valued fields.

Gene: stores standardized gene-level information, such as the gene symbol, gene identifier, and chromosome location. Entrez ID and HUGO symbol both served as gene identifiers. Entrez ID is more stable than HUGO symbol but gene symbol is more human readable, and also this design of having two gene identifiers can help with avoiding removal of too many data because as long as an entry for the data has at least 1 gene identifier, the gene related to that entry is identifiable. One thing to note is that the table has the surrogate primary key gene_id instead of utilizing Entrez ID as the primary key. This decision was made to solve the problem of gene entries that only have the HUGO symbol recorded.

Expression: stores gene expression value for each sample and gene. This was done because expression data depends on both the sample and the gene: one sample can have expression values for many genes, and one gene can be measured across many samples.

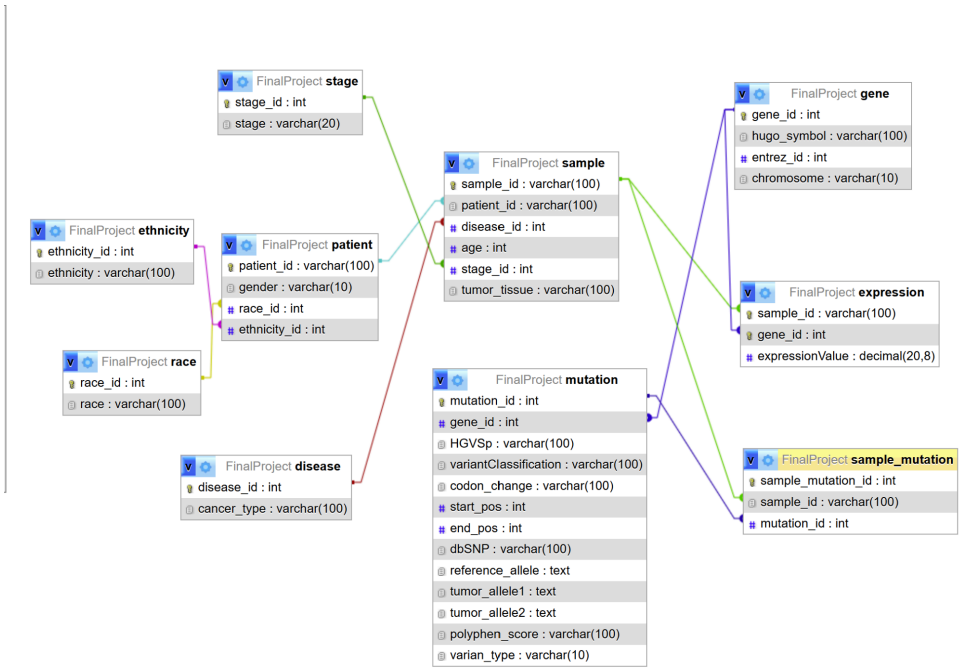

Figure 2. A complete database design

Table	Action	Rows	Type	Collation	Size	Overhead
disease	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
ethnicity	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
expression	Browse Structure Search Insert Empty Drop	7,489,824	InnoDB	utf8mb4_0900_ai_ci	290.5 MiB	-
gene	Browse Structure Search Insert Empty Drop	20,887	InnoDB	utf8mb4_0900_ai_ci	1.5 MiB	-
mutation	Browse Structure Search Insert Empty Drop	56,107	InnoDB	utf8mb4_0900_ai_ci	8.0 MiB	-
patient	Browse Structure Search Insert Empty Drop	372	InnoDB	utf8mb4_0900_ai_ci	80.0 KiB	-
race	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
sample	Browse Structure Search Insert Empty Drop	372	InnoDB	utf8mb4_0900_ai_ci	144.0 KiB	-
sample_mutation	Browse Structure Search Insert Empty Drop	56,846	InnoDB	utf8mb4_0900_ai_ci	6.5 MiB	-
stage	Browse Structure Search Insert Empty Drop	9	InnoDB	utf8mb4_0900_ai_ci	16.0 KiB	-
10 table(s)	**Sum**	**7,624,424**	**InnoDB**	**utf8mb4_0900_ai_ci**	**306.8 MiB**	**0 B**

Figure 3. Database after data are populated

Normal form and deviations

The database is mostly normalized and was designed to follow Third Normal Form. Each table was created to represent one main entity such as patient, sample, disease, stage, gene, mutation, expression. Normalization choices to achieve the First Normal Form (1NF) were that each table have a primary key to allow each record to be uniquely identified, each field stores only one value, and lists of values are not stored in one column. Normalization choices to achieve the Second Normal Form (2NF) were making sure that non-key attributes depend on the whole primary key and avoiding partial dependency. Examples of these are the expressionValue in the

expression table depends on both sample_id and gene_id, and sample attributes or mutation attributes were not placed in the sample_mutation table because those details only depend on part of the relationship. Normalization choices to achieve Third Normal Form were making sure that non-key attributes are not dependent on other non-key attributes. Repeated categorical values are stored in separate tables. For example, race and ethnicity have their own tables and are referenced by the patient table using foreign keys. Similarly, cancer type and cancer stage are stored in the disease and stage tables instead of being stored repeatedly in the sample table. Gene-level details like HUGO symbol, Entrez ID, and chromosome are stored in the gene table and referenced by mutation and expression tables through gene_id. This helps reduce transitive dependency, avoids storing repeated categorical data, and improves consistency across the database.

The current design has no major deviation from normal form. Some designs that could be considered questionable are storing age in the sample table, and small tables such as race, ethnicity, stage, and disease. As addressed in the Database design decision section, a patient can have multiple cancer diagnoses at different ages in their lifetime so age is more appropriate to link to the sample table to indicate the age of the patient at the time the sample was analyzed. This could help with data integrity if the data is used to study the relationship between mutation and age. Small tables like race, ethnicity, stage, and disease might be simple and seem to be over-normalized but these tables avoid redundancy and ensure consistency if there is an update to any of the categorical values. One thing that the design could improve on is using a composite key in the sample_mutation table from sample_id and mutation_id instead of generating sample_mutation_id.

Relationships and constraints

The database uses primary keys to uniquely identify records in each table and foreign keys to enforce relationships between entities.

List of primary keys

- Patient_id: uses PATIENT_ID from data
- race_id
- ethnicity_id
- sample_id: uses SAMPLE_TUMOR_BARCODE from data file
- disease_id
- stage_id
- gene_id
- mutation_id
- sample_mutation_id
- Composite key: sample_id + gene_id

The database uses foreign keys to connect related tables and make sure that data being referenced from other tables is valid. In this database, foreign keys help connect patient demographics, cancer samples, disease type, cancer stage, genes, mutations, and expression values. For example, the foreign key patient_id in the sample table links each sample to a valid

patient. In the expression table, a composite key of sample_id and gene_id helps identify expression value because the expression value depends on the gene and the sample. One sample can have multiple expression values across genes, and one gene can have multiple expression values across samples so creating a composite key for expression value is the best way to keep the expression record unique. Constraints for the database is that each table should have a unique identifier (primary key constraint), foreign key constraint make sure records connect to valid records from other tables, and other attributes such as polyphen_score, or gender can be null but primary keys cannot be null.

Data dictionary

Table: patient

Field Name	Data type	Key/Constraint	Description
patient_id	VARCHAR(100)	Primary Key, Not Null	Unique identifier for each patient from the data's PATIENT_ID.
gender	VARCHAR(10)	Nullable if unknown	Patient's recorded gender.
race_id	INT	Foreign Key → race(race_id)	Links the patient to a race category.
ethnicity_id	INT	Foreign Key → ethnicity(ethnicity_id)	Links the patient to an ethnicity category.

Table: race

Field Name	Data type	Key/Constraint	Description
race_id	INT	Primary Key, Not Null	Unique identifier for each race category.
race	VARCHAR(100)	Unique, Not Null	Race category from data file.

Table: ethnicity

Field Name	Data type	Key/Constraint	Description
ethnicity_id	INT	Primary Key, Not Null	Unique identifier for each ethnicity category.
ethnicity	VARCHAR(100)	Unique, Not Null	Ethnicity category from data file.

Table: sample

Field Name	Data type	Key/Constraint	Description
sample_id	VARCHAR(100)	Primary Key, Not Null	Unique identifier for each tumor sample.
patient_id	VARCHAR(100)	Foreign Key → patient(patient_id), Not Null	Links the sample to the patient.
disease_id	INT	Foreign Key → disease(disease_id), Not Null	Links the sample to a cancer type.
age	INT	Nullable if unknown	Patient age at the time of sample collection or record.

stage_id	INT	Foreign Key → stage(stage_id)	Links the sample to a cancer stage.
tumor_tissue	VARCHAR(150)	Nullable if unknown	Tumor source description.

Table: disease

Field Name	Data type	Key/Constraint	Description
disease_id	INT	Primary Key, Not Null	Unique identifier for each disease or cancer type
cancer_type	VARCHAR(100)	Unique, Not Null	Standardized cancer type name

Table: stage

Field Name	Data type	Key/Constraint	Description
stage_id	INT	Primary Key, Not Null	Unique identifier for each cancer stage
stage	VARCHAR(20)	Unique, Not NulL	Cancer stage label: I, II, III, IIIA, IIIB, IIIC, IV, IVA, IVB

Table: gene

Field Name	Data type	Key/Constraint	Description
gene_id	INT	Primary Key, Not Null	Internal unique identifier for each gene.
hugo_symbol	VARCHAR(100)	Unique but nullable if unknown	Standard HUGOgene symbol, such as TP53, BRCA1, or EGFR
entrez_id	INT	Unique but nullable if unknown	External Entrez gene identifier.
chromosome	VARCHAR(10)	nullable	Chromosome where the gene is located.

Table: mutation

Field Name	Data type	Key/Constraint	Description
mutation_id	INT	Primary Key, Not Null	Unique identifier for each mutation record.
gene_id	INT	Foreign Key → gene(gene_id), Not Null	Links the mutation to the gene where it occurs.
HGVSp	VARCHAR(100)	Nullable	Protein-level mutation notation.
variantClassification	VARCHAR(100)	Nullable	Functional classification of the mutation.
codon_change	VARCHAR(100)	Nullable	Codon-level change caused by the mutation.
start_pos	INT	Nullable	Genomic start position of the mutation.
end_pos	INT	Nullable	Genomic end position of the mutation.

reference_allele	TEXT	Nullable	Reference genome allele.
tumor_allele1	TEXT	Nullable	First observed tumor allele.
tumor_allele2	TEXT	Nullable	Second observed tumor allele.
polyphen_score	VARCHAR(100)	Nullable	Predicted effect score of the mutation.
varian_type	VARCHAR(10)	Nullable	General type of mutation or variant.
dbSNP	VARCHAR(100)	Nullable	dbSNP reference identifier

Table: sample_mutation

Field Name	Data type	Key/Constraint	Description
sample_mutation_id	INT	Primary Key, Not Null	Unique identifier for each sample-mutation relationship.
sample_id	VARCHAR(100)	Foreign Key → sample(sample_id), Not Null	Links the record to a sample.
mutation_id	INT	Foreign Key → mutation(mutation_id), Not Null	Links the record to a mutation.

Table: expression

Field Name	Data type	Key/Constraint	Description
sample_id	VARCHAR(100)	Composite Primary Key, Foreign Key → sample(sample_id)	Identifies the sample for the expression measurement.
gene_id	INT	Composite Primary Key, Foreign Key → gene(gene_id)	Identifies the gene for the expression measurement.
expressionValue	DECIMAL(20,8),	Not Null	Measured expression value for a specific gene in a specific sample.

Script map

The table below shows the order of running the scripts, the description of each script, the files that use the script and the output files.

Python scripts

Order of usage	Script name	Description	Files that use the script	Output files
1	01_clean_data.py	This Python script performs initial data cleaning for all files used in the database. It checks for duplicate records, replacing missing-value placeholders such as . with empty values, converts raw .txt files to .tsv format, and removes columns that have headers but no data. This step should be run before importing the files into the	data_mrna_seq_v2_rsem.txt data_clinical_patient.txt data_clinical_sample.txt data_mutations.txt	expression_cleaned.tsv patient_data_cleaned_final.tsv sample_data_cleaned_final.tsv data_mutation_cleaned.tsv

		database to make later cleaning and loading easier.		
2	02_map_genes.py	**Reference file used in the script:** hgnc_complete_set.txt This script updates outdated HUGO symbols or missing HUGO symbol/Entrez ID in the mutation file(mutation_cleaned.tsv) and expression file (expression_cleaned.tsv) by looking up the HUGO symbol/Entrez ID in these files and matches it with the reference file. If the symbol/ID in data files matches with the symbol/ID that is considered outdated in the reference file, the previous symbol/ID columns in the expression and mutation file will be overwritten. The script also adds additional columns containing original symbol/ID, updated symbol/ID, update based on Entrez ID or HUGO symbol. This is so we can verify that updates have been made and the number of updates the script made. Note: symbol/ID that matches more than one entries in a reference file is considered ambiguous and is not updated.	expression_cleaned.tsv data_mutation_cleaned.tsv	expression_mapped_final.tsv mutation_mapped.tsv
3	03_fix_mutation_from_expression.py	**Reference file used in the script:** expression_mapped_final.tsv The mutation_mapped.tsv file still has some entries with invalid Entrez IDs after mapping using a reference document. The expression_mapped_final.tsv file has no invalid Entrez IDs so this was used by the script to do a final mapping on the mutation file to map as many Entrez IDs as possible. The Entrez IDs that were not mapped to a valid ID will be replaced with appropriate null value instead of 0. This is to ensure the empty fields are unified across data files. Note: symbol/ID that matches more than one entry in a reference file is considered ambiguous and is not updated.	mutation_mapped.tsv	mutation_mapped_fixed_final.tsv
4	04_write_sql.py	This Python script prepares the cleaned clinical cancer and genomic data for database loading. It reads the final cleaned patient, sample, mutation, and expression TSV files, standardizes column names, checks for duplicate headers, and maps source columns to the column names used in the database schema. The script creates lookup IDs for race, ethnicity, disease, stage, gene, and mutation records, then generates SQL INSERT statements for each database table. It also transforms the expression file from wide format into long format so each row represents one expression value for one sample and one gene. Finally, it creates bridge-table records for sample-mutation relationships and writes all SQL commands to 02_load_cleaned_data.sql, which can be run to populate the database.	mutation_mapped_fixed_final.tsv expression_mapped_final.tsv patient_data_cleaned_final.tsv sample_data_cleaned_final.tsv	02_load_cleaned_data.sql

SQL

Order of usage	Script name	Description
1	01_create_schema.sql	Create the database and tables according to the ER diagram.
2	02_load_cleaned_data.sql	Script was generated using 04_write_sql.py. This helps load cleaned data from data files to the database. This is the last file to run to populate the data into the database.
N/A	database_dump	Script to recreate, back up, or transfer a database. This script does not need to be run if the other two sql files were run to build database.

Reproduction instruction

Software needed: Oracle VirtualBox version 7.2.0 or local XAMPP like Apache, phpMyAdmin, Python 3.13, PyCharm 2025.2.2 or other Integrated Development Environment (IDE).

1. Install Oracle VirtualBox or local XAMPP. This project used Oracle VirtualBox version 7.2.0. Install Python and an IDE. The IDE used in this project was PyCharm 2025.2.2 and Python 3.13.
2. Retrieve the raw data files from GitHub repository or from cBioPortal using the following link: https://www.cbioportal.org/study/summary?id=lihc_tcga_pan_can_atlas_2018
3. Run the first python script `01_clean_data.py` on `data_clinical_patient.txt` (patient data) to get `patient_data_cleaned_final.tsv`. The python script has the file name hardcoded so the name of the input file and the output file needs to be changed after every run.
4. Run the first python `01_clean_data.py` on `data_clinical_sample.txt` (sample data) to get `sample_data_cleaned_final.tsv`
5. Run the first python `01_clean_data.py` on `data_mutations.txt` (mutation data) to get `data_mutations_cleaned.tsv`
6. Run the first python `01_clean_data.py` on `data_mrna_seq_v2_rsem.txt` (expression data) to get `expression_cleaned.tsv`
7. Use `hgnc_complete_set.txt` as reference file when running the second python script `02_map_genes.py` on both `data_mutations_cleaned.tsv` and `expression_cleaned.tsv` at the same time to get `mutation_mapped.tsv` and `expression_mapped_final.tsv`
8. Run the third python script `03_fix_mutation_from_expression.py` on `mutation_mapped.tsv` and use `expression_mapped_final.tsv` as reference to get `mutation_mapped_fixed_final.tsv`
9. Run the fourth python script `04_write_sql.py` on all `mutation_mapped_fixed_final.tsv, expression_mapped_final.tsv,`

`sample_data_cleaned_final.tsv, patient_data_cleaned_final.tsv` to get the `02_load_cleaned_data.sql.`

10. Install mysql using this following command in the terminal >`sudo dnf install mysql`
11. Run the following command to access the database `>mysql -h 127.0.0.1 -P 3306 -u root -pinstructor`
12. Run the first SQL file (`01_create_schema.sql)` to create the database and tables in the database. To execute the SQL file, run the following command in the terminal, replace shared_folder_name with the actual shared folder name.
    ```
    > mysql -h 127.0.0.1 -P 3306 -u root -pinstructor
    /media/shared_folder_name/01_create_schema.sql
    ```
13. Run the second SQL file (`02_load_cleaned_data.sql`) to load data into the database. To execute the SQL file, run the following command in the terminal, replace shared_folder_name with the actual shared folder name.
    ```
    > mysql -h 127.0.0.1 -P 3306 -u root -pinstructor
    /media/shared_folder_name/02_load_cleaned_data.sql
    ```
14. Access the database through PHPMyAdmin by navigating to http://localhost:8888/

Limitations and future work

The first limitation about this database is that the database structure only allows one disease per sample when in reality there could be multiple disease links to one sample. The current design of the database works for this dataset because the dataset contains data only on liver cancer. If this database expands to store other diseases, this design issue would need to be addressed by adding a disease_sample junction table to allow many-to-many relationships between disease and sample. One small detail that the database design could have improved on is to use a composite key for the sample_mutation table to avoid duplications. Another limitation is that the python scripts to clean data and map genes are hardcoded so the file name needs to be changed in the file whenever running the script if the file name is different from the file name in the script. Expression data in this database is raw data so the data can be more flexible to different types of downstream analyses, however, the raw data is less interpretable than normalized data. The amount of data included in this database was minimal compared to the information that the whole dataset covered. The minimal database design was chosen with intention to save time when executing the scripts, uploading the data into the database and in consideration of limited storage space. There are files including links to pathology reports, images, or protein data and ancestry data that would have been useful for research if incorporated into the database. Despite the minimal design of the database, the amount of data used was still considered big so it is possible that there are errors or duplications in the database because the data cleaning and checking did not cover those errors.

Future work on this database would include expanding the database design so it could contain protein information, ancestry data, and possibly images from diagnosis to improve the usefulness

of the database. In addition, expanding to include other types of cancer is also a potential future work for this database.

Query examples:

patient_id	sample_id	gene_id	hugo_symbol	entrez_id	chromosome	mutation_id	HGVSp	variantClassification	codon_change	start_po
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	21	AAMP	14	2	47	p.*14*	5'UTR	NULL	219134
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	141	ACAP1	9744	17	610	p.P263H	Missense_Mutation	cCt/cAt	7247
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	750	ANKRD20A4P	728747	9	2636	p.E183K	Missense_Mutation	Gaa/Aaa	693899
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	992	ARFGAP3	26286	22	3275	p.*49*	5'UTR	NULL	43253
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1147	ARRB1	408	11	3820	p.M255T	Missense_Mutation	aTg/aCg	749878
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1201	ASCC1	51008	10	3921	NULL	3'UTR	NULL	73857
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1296	ATOH8	84913	2	4241	p.S166=	Silent	tcA/tcG	859818
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1513	BAP1	8314	3	4853	p.M115Rfs*9	Frame_Shift_Del	aTGAAG/a	52442
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1748	BPTF	2186	17	5445	p.X2029_splice	Splice_Site	NULL	65920
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1775	BRINP3	339479	1	5556	p.R532=	Silent	Cgg/Agg	190067
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1800	BTAF1	9044	10	5636	p.*13*	5'UTR	NULL	93683
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1971	C2	717	6	5945	NULL	3'Flank	NULL	31914
TCGA-2Y-	TCGA-2Y-A9H6-01	2123	CACNA1G	8913	17	6369	p.D1502=	Silent	gaC/gaT	48684

Console

Figure 4a. Looking up mutation information using patient_id

patient_id	sample_id	gene_id	hugo_symbol	entrez_id	chromosome	expressionValue
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	1	A1BG	1	19	22432.90000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	2	A1BG-AS1	503538	19	162.52500000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	3	A1CF	29974	10	1004.80000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	4	A2M	2	12	276645.00000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	5	A2M-AS1	144571	NULL	242.90100000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	6	A2ML1	144568	12	5.90620000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	7	A4GALT	53947	22	157.25400000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	8	A4GNT	51146	3	4.42970000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	9	AAAS	8086	NULL	893.31900000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	10	AACS	65985	12	428.20200000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	11	AACSP1	729522	5	0.00000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	12	AADAC	13	3	6166.85000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	13	AADACL2	344752	3	0.00000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	14	AADACL3	126767	1	0.00000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	15	AADACL4	343066	1	0.00000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	16	AADACP1	201651	NULL	63.49210000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	17	AADAT	51166	4	102.62100000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	18	AAGAB	79719	NULL	1037.28000000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	19	AAK1	22848	2	847.54500000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	20	AAMDC	28971	NULL	555.18600000
TCGA-2Y-A9H6	TCGA-2Y-A9H6-01	21	AAMP	14	2	3198.97000000
Console 9H6	TCGA-2Y-A9H6-01	22	AANAT	15	17	0.00000000

Figure 4b. Looking up mutation information using patient_id

```
SELECT DISTINCT p.patient_id, s.sample_id, m.dbSNP, m.variantClassification, m.HGVSp, -- gene information g.gene_id, g.hugo_symbol, --
expression information for the same gene/sample e.expressionValue FROM mutation m JOIN sample_mutation sm ON m.mutation_id =
sm.mutation_id JOIN sample s ON sm.sample_id = s.sample_id JOIN patient p ON s.patient_id = p.patient_id JOIN gene g ON m.gene_id =
g.gene_id LEFT JOIN expression e ON s.sample_id = e.sample_id AND m.gene_id = e.gene_id WHERE m.dbSNP = 'rs1395505056';
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

patient_id	sample_id	dbSNP	variantClassification	HGVSp	gene_id	hugo_symbol	expressionValue
TCGA-2V-A95S	TCGA-2V-A95S-01	rs1395505056	Missense_Mutation	p.P697S	3486	COL4A1	2198.40000000

Figure 4c. Looking up patient_id, sample_id, expression value and some mutation information using SNP.